

Доставка уведомлений — архитектура

Дата: 2026-08-16 · **Для кого:** бэкенд-разработчики и члены команды, которым нужно верхнеуровневое понимание того, что куда ходит.

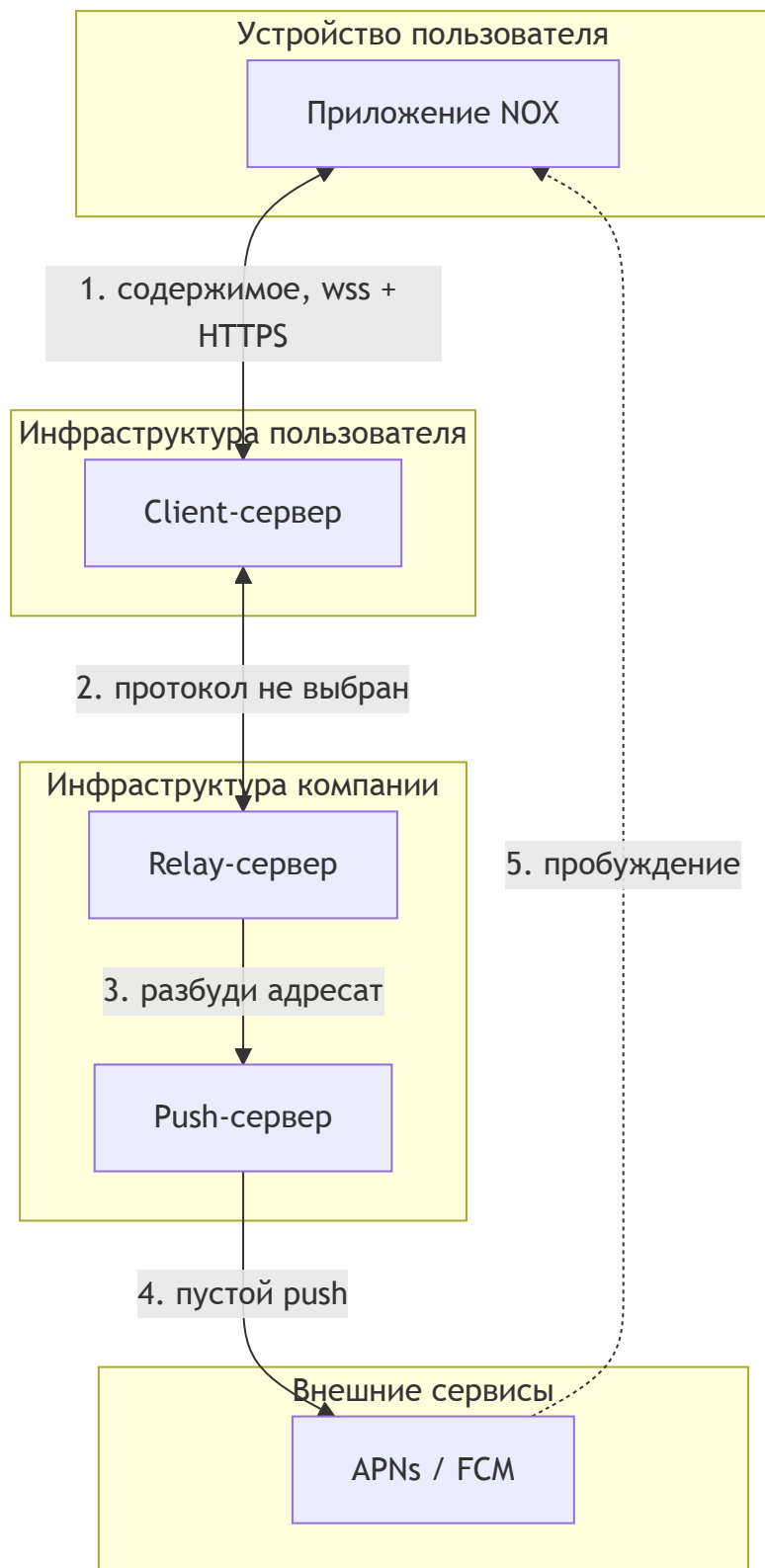
Основано на [notification-delivery-solution.md](#) — там обоснования, первоисточники и отвергнутые варианты. Здесь только схема работы.

⚠ **Протокол «приложение ↔ client-сервер» и «client-сервер ↔ relay» ещё не выбран (Q13).** Поэтому пакеты на этих участках описаны **по смыслу**, а не по байтам. Форматы APNs и FCM, наоборот, заданы Apple и Google и приведены буквально.

1. Действующие лица

Компонент	Чей	За что отвечает
Приложение	устройство пользователя	Показывает уведомление, ведёт догоняющую синхронизацию
Client-сервер	пользователь, разворачивает сам	Хранит сообщения, отдаёт «что нового с курсора X», сообщает вверх о событии
Relay-сервер	компания	Связывает client-серверы между собой. Про push знает только «надо разбудить вот этот непрозрачный адресат»
Push-сервер	компания, отдельно от relay	Единственный держатель ключей APNs и FCM. Больше не делает ничего
APNs / FCM	Apple / Google	Переносят сигнал пробуждения

2. Топология



Три границы, которые нельзя перепутать:

- Приложение разговаривает **только со своим client-сервером**. Ни про relay, ни про push-сервер оно не знает и адресов их не имеет.
- **Содержимое** ходит исключительно по стрелке 1. По стрелкам 3–5 не идёт ничего, кроме факта «разбуди».

- Стрелка 5 — пунктиром, потому что доставка best-effort: гарантии нет (см. §7).

3. Почему push-сервер отделён от relay

Разделение введено решением владельца ради операционной развязки: relay не должен зависеть от инфраструктуры Apple и Google, а push-сервер не должен знать ничего о смысле сообщений.

Побочный эффект оказался важнее самой развязки. Разнеся эти две роли, мы бесплатно получаем свойство, которое иначе пришлось бы проектировать отдельно: **ни один из двух серверов компании в одиночку не может связать «кому» с «что».**

	Токен устройства	Что произошло	Содержимое
Client-сервер	 только в зашифрованном виде	 да	 да
Relay-сервер	 нет , непрозрачный blob	 факт и время	 нет
Push-сервер	 да, расшифровывает	 нет	 нет
APNs / FCM	 да	 нет	 нет

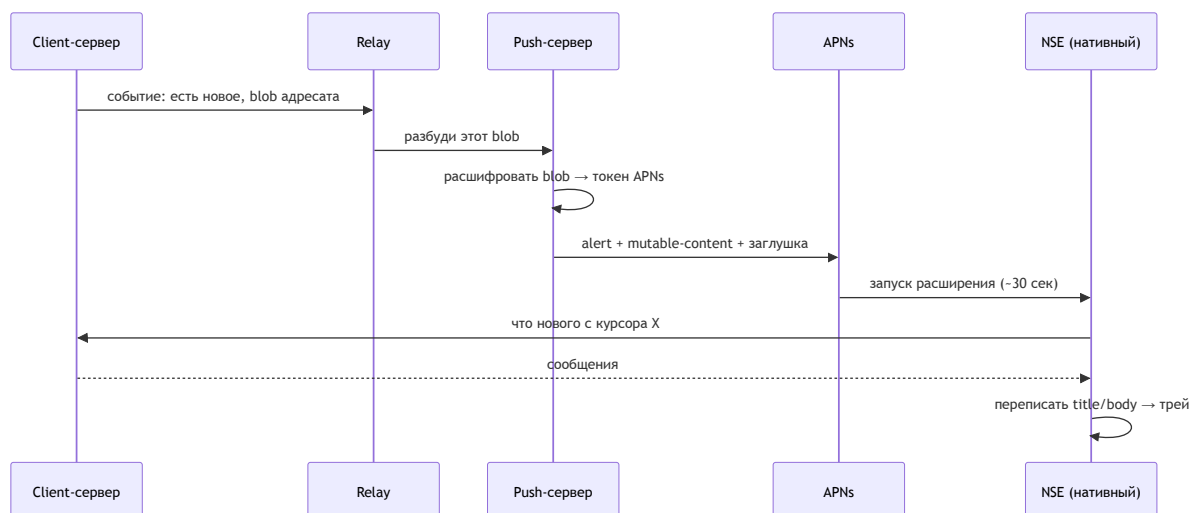
Механика: **токен устройства шифруется ключом push-сервера ещё на устройстве.** Client-сервер и relay передают его как непрозрачный blob и расшифровать не могут. Это модель chatmail (Delta Chat), она уже была рекомендована в [Q2](#) — теперь она совпала с формой, которую владелец выбрал по другим причинам.

 **Честная оговорка:** оба сервера наши. Свойство защищает от компрометации *одного* из них и от нелюбопытного оператора, но не от сговора или изъятия обоих. Продавать это как гарантию нельзя.

4. Платформы: три разных пути

	Как будим	Кто идёт за содержимым	Кто рисует уведомление
iOS	alert-push с mutable-content	Notification Service Extension, нативный Swift	iOS, подменённым текстом
Android	data-only FCM, high priority	фоновый Dart-изолят	приложение, локальное уведомление
Десктопы	никак — push не используется	сам резидент по сокету	приложение, локальное уведомление

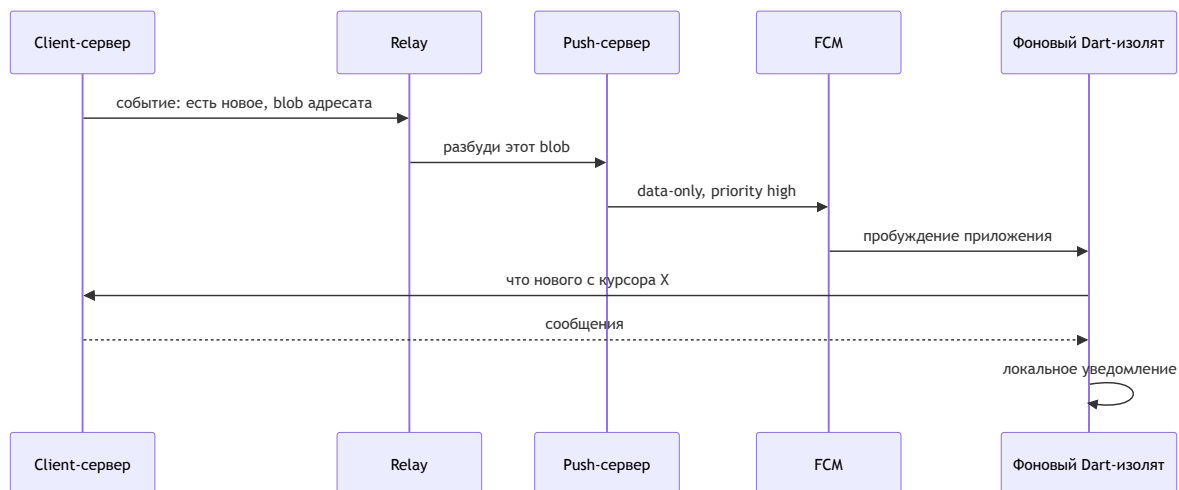
5. iOS



❗ Если шаг «что нового» не уложился в ~30 секунд или сервер недоступен — iOS показывает заглушку из payload. Она и есть UX отказа, поэтому её текст выбирается осознанно (у Signal — «You may have new messages»).

⚠ Расширение — отдельный нативный процесс, Dart в нём не работает. Клиентская часть протокола и криптоядро должны быть вызываемы из Swift (Q10).

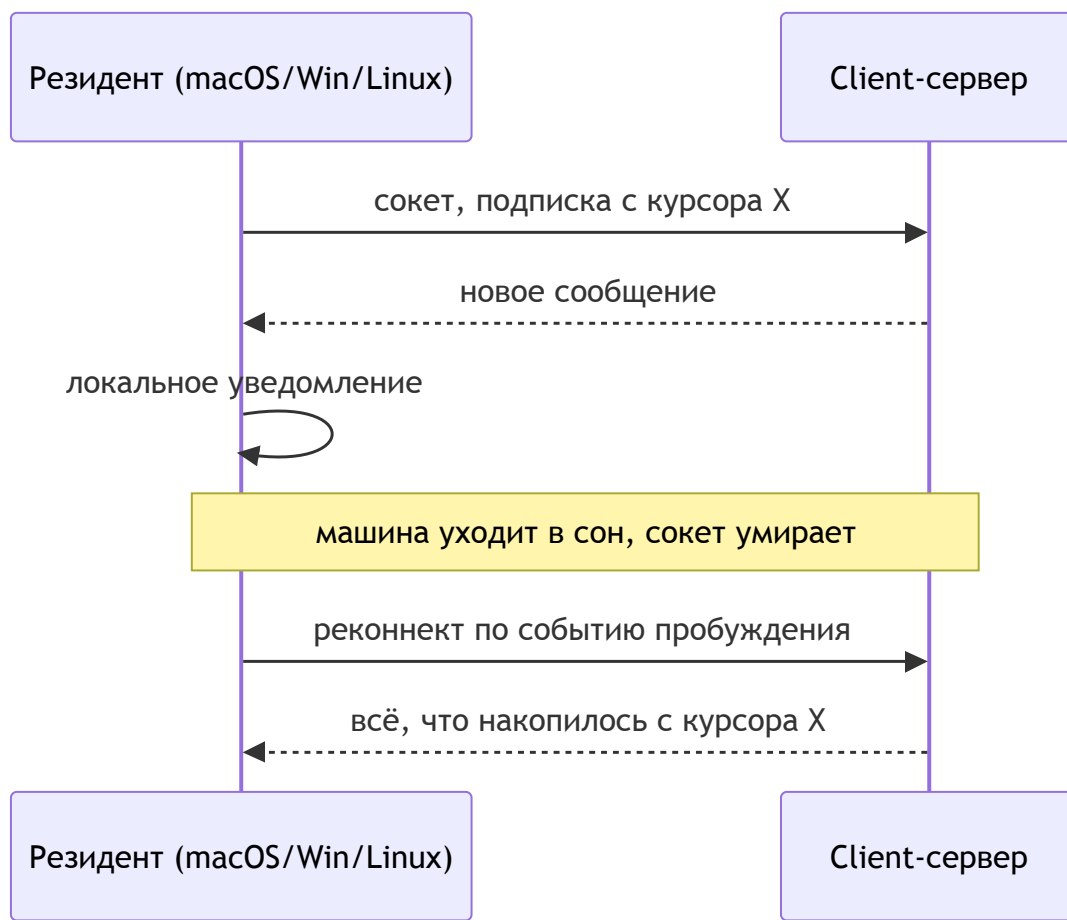
6. Android



⚠ Сообщение должно быть **data-only**: если положить в него блок `notification`, FCM в фоне отдаст его прямо в трей и наш код не вызовет — содержимое взять будет неоткуда.

⚠ FCM понижает приоритет отправителям, чьи push систематически не заканчиваются видимым уведомлением (окно — скользящие 7 дней). Схема безопасна ровно потому, что уведомление в итоге показывается.

7. Десктопы — push не участвует вовсе



Состояния, которые должен реализовать клиент:



ОС	Автозапуск	Событие пробуждения
macOS	SMAppService	NSWorkspace.didWakeNotification
Windows	Task Scheduler по входу в систему	PBT_APMRESUMEAUTOMATIC
Linux	systemd user unit, Restart=always	D-Bus PrepareForSleep(false)

🟡 **Windows:** в Modern Standby на батарее сетевой стек сам отключает Wi-Fi и Ethernet — сокет не живёт, уведомления приезжают при пробуждении. На зарядке живёт.

8. Какие пакеты ходят

#	Откуда → куда	Когда	Что внутри	Чего внутри нет
1	Приложение → свой Client-сервер	первый запуск и ротация токена	Токен устройства, зашифрованный ключом push-сервера ; тип платформы	—
2	Client-сервер → Relay	появилось новое для устройства	Непрозрачный blob адресата; тип платформы	Текст, отправитель, имя чата
3	Relay → Push-сервер	немедленно	Команда «разбуди»; тот же blob	Что произошло, от кого
4	Push-сервер → APNs / FCM	немедленно	См. ниже — формат задан вендором	Любое содержимое
5	APNs / FCM → устройство	best-effort	То же	—
6	Устройство → свой Client-сервер	после пробуждения	«Что нового с курсора X»	—
7	Client-сервер → устройство	в ответ	Сообщения	—

Участки **1, 6, 7** — наш протокол, ещё не выбран. Участок **2** — Q13. Участки **4–5** зафиксированы вендорами:

iOS (APNs) — обязательны оба условия, иначе расширение не запустится:

```
POST /3/device/<token>
apns-push-type: alert
apns-priority: 10
apns-topic: <bundle id>

{ "aps": { "alert": { "body": "<заглушка>" },
  "mutable-content": 1 } }
```

Android (FCM) — блока `notification` быть не должно:

```
POST /v1/projects/<id>/messages:send

{ "message": { "token": "<token>",
  "android": { "priority": "high" },
  "data": { "v": "1" } } }
```

9. Что из этого следует для бэкенда

Client-сервер обязан:

1. Отдавать «**что изменилось с курсора X**», а не «последние N сообщений». Это не удобство, а необходимость: и десктоп после сна, и мобильный после пропущенного push догоняют именно так. Модель «отдай последнее» будет терять либо дублировать.
2. Хранить зашифрованный токен устройства, не пытаясь его прочитать.
3. Порождать событие наверх при появлении нового — по одному на устройство-адресат.

Push-сервер обязан:

1. Быть единственным местом, где лежат ключи APNs и FCM. **Не копировать их в client-серверы** — обладатель ключа может слать push всем пользователям NOX.
2. Расшифровывать blob и вызывать нужного вендора по типу платформы.
3. Не хранить ничего о смысле события.

Общее правило, вытекающее из отсутствия гарантий доставки:

Уведомление — это оптимизация, а не транспорт. Единственный гарантированный момент синхронизации — когда пользователь сам открыл приложение. Пропущенный push не должен означать потерянное сообщение.

10. Открытые вопросы

	Вопрос	Реестр
●	Язык и форма криптоядра — должно вызываться из Swift внутри расширения	Q10
●	Как устройство регистрирует токен и как ключ push-сервера попадает на устройство	открыт, отдельно
●	Протокол «client-сервер ↔ relay»	Q13
●	ATS и самоподписанные пользовательские серверы на iOS	Q14
●	Текст заглушки в payload — его видит пользователь при отказе	—
●	Windows: MSIX ради тостов и <code>SocketActivityTrigger</code>	—